

TECHCATALYST DE 2026

Data at Rest

Object storage, analytical structures, lifecycle, and query-in-place.

The Hartford · TechCatalyst Data Engineering 2026

Week 1 Day 4

Architectures & separation

Formats & landing zones

Lifecycle & protection

Query-in-place

Hands-on labs

WHY THIS MATTERS

Data at rest is an engineered layer

Durable foundation

Pipelines need a reliable place to land raw data before every use is known.

Shared contract

Names, formats, schemas, and zones let teams and engines interpret the same data.

Cost and risk

Access tiers, IAM, recovery, and retention determine operating cost and exposure.

The bucket is only the start. Organization, controls, metadata, and compute make stored data useful.

DAY MAP

Place → organize → protect → query → operate

Place and compare

1. Place object storage in a pipeline.
2. Compare lake, lakehouse, warehouse, and mart.

Organize and protect

3. Explain storage/compute separation.
4. Choose formats, prefixes, zones, and lifecycle tiers.

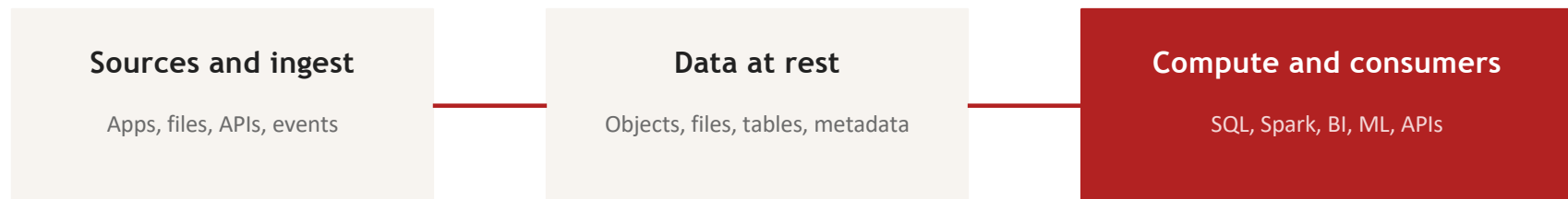
Query and operate

5. Distinguish recovery and retention controls.
6. Build and query a secure landing zone.

Today builds the storage contract that tomorrow's NYC Taxi pipeline will use.

PIPELINE VIEW

Data at rest connects movement to use



- 1. Ingest moves data; storage preserves it.
- 2. Multiple engines may read the same durable data.
- 3. Layout, schema, IAM, and lifecycle travel with the data contract.
- 4. Day 5 adds pipeline movement around this stored layer.

Why Big Data changed architecture

Volume

Too much data for one machine or one expensive appliance.

Variety

Tables, JSON, logs, images, PDFs, audio, video, and embeddings.

Velocity

Nightly batches plus events arriving continuously.

Veracity

Bad, late, duplicated, and drifting data can damage decisions.

Value

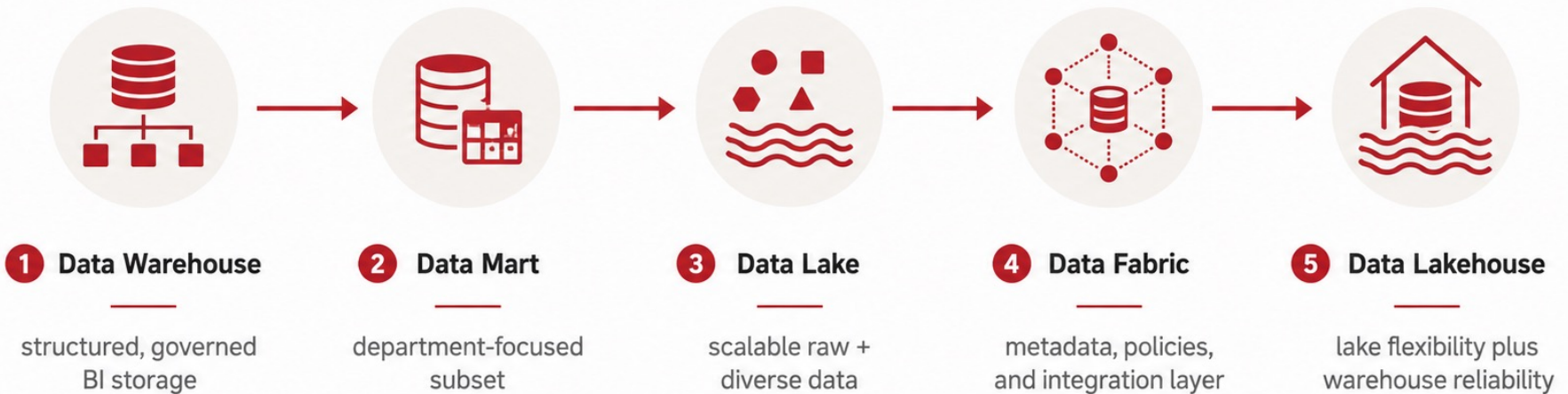
Storage is not the goal. Reliable use by BI, ML, apps, and humans is the goal.

The rest of today answers one question: where do we put varied data so it is durable, governable, affordable, and useful later?

ARCHITECTURE EVOLUTION

From warehouse to lakehouse

The evolution of data-at-rest architectures



Each step expands data variety, accessibility, governance, and analytical capability.

FOUNDATIONS

Data Warehouse

Centralized, curated storage for structured reporting and BI.



What it is

A centralized repository of integrated, structured data optimized for reporting and decision support.



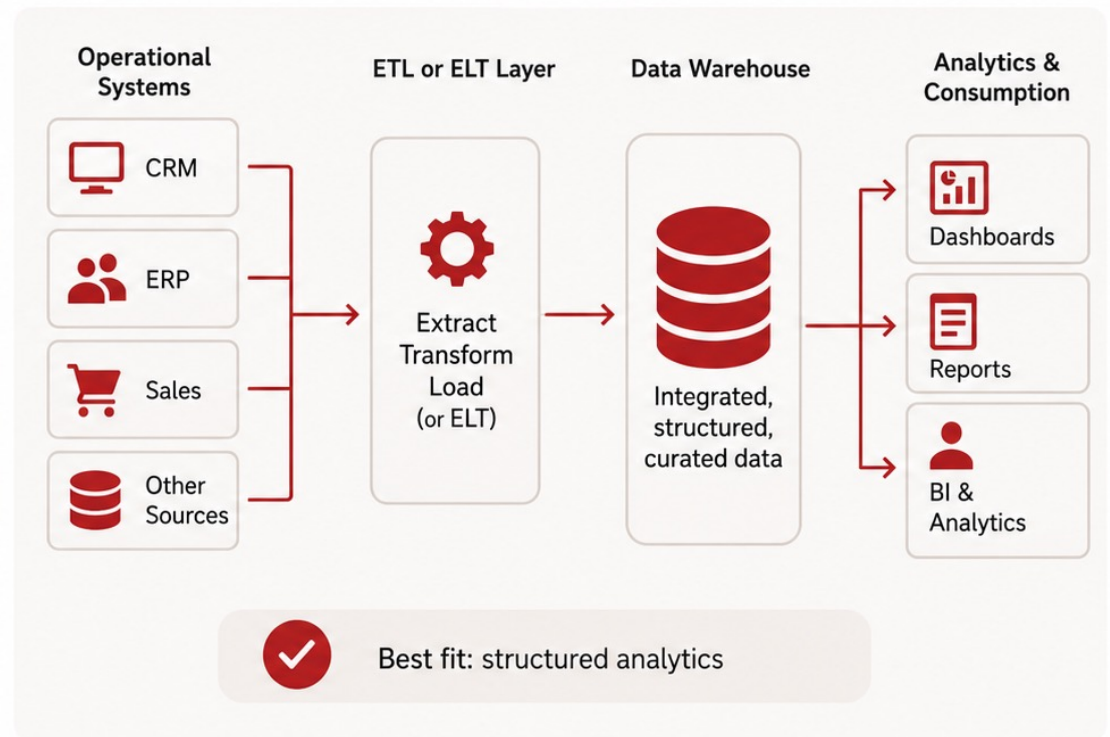
Strengths

Trusted metrics, historical analysis, schema-on-write, strong governance.



Limitations

Less flexible for semi-structured or unstructured data and exploratory ML workloads.



SUBJECT-AREA ANALYTICS

Data Mart

A focused subset of enterprise data designed for a team, function, or business domain.



Purpose

Provides targeted, faster access to curated data for a specific department such as Finance, HR, or Marketing.



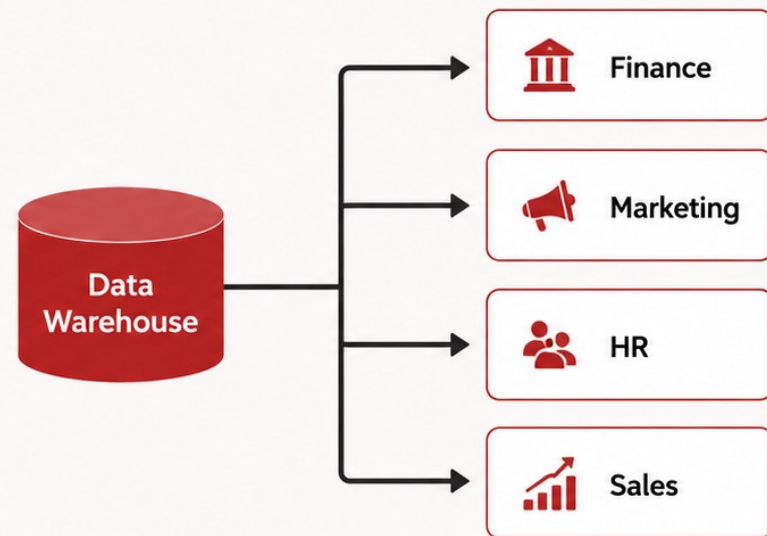
Benefits

Simpler access, domain-focused metrics, better usability for business teams.



Trade-offs

If unmanaged, multiple marts can duplicate data and create silos.



Think of it as a department-ready view of enterprise data.

OPEN STORAGE

Data Lake

Scalable, low-cost storage for structured, semi-structured, and unstructured data.



What it is:

A storage layer that keeps data in open file formats and preserves it for many analytical uses.



Strengths:

Handles many data types, supports schema-on-read, and works with BI, SQL, Spark, and machine learning.



Challenges:

Without governance and quality controls, lakes can become hard-to-use “data swamps”.



Key idea: store first, model later.

SOURCES



Files

Parquet, ORC, JSON, CSV



Databases

On-prem and cloud



APIs

Applications and services



IoT devices

Sensors and telemetry



Logs & events

Applications and systems



Audio / video

Media and documents

STORE ONCE ACCESS MANY

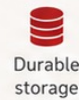


Data Lake

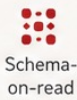
Open file formats in scalable, low-cost storage



Open formats



Durable storage



Schema-on-read

CONSUMERS



SQL analytics

Query and report



Data science

Explore and analyze



Machine learning

Train and predict



Dashboards & BI

Visualize and share



Governance and access



Quality and cataloging



Lifecycle and cost controls

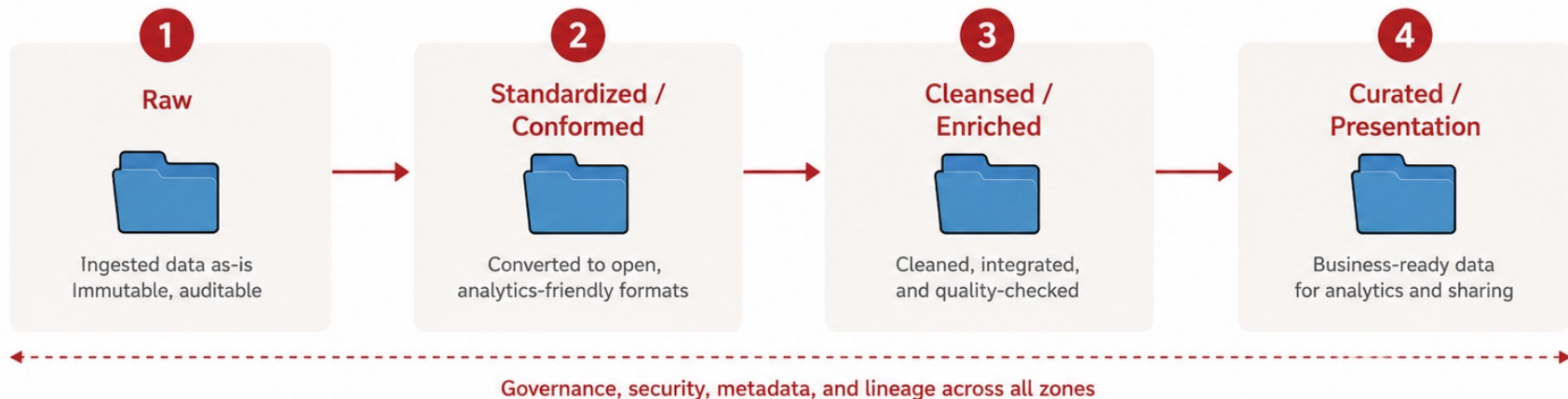


Observability and lineage

LAKE DESIGN

Data lake zones and best practices

Organize raw data for trust, reuse, and downstream analytics.



- ✓ Keep the raw zone immutable for auditability and replay.
- ✓ Convert to open, analytics-friendly formats such as Parquet.
- ✓ Organize paths by domain, source, object, and date.
- ✓ Apply governance, quality checks, and clear ownership to avoid a data swamp.

Example path patterns

 /raw/sales/orders/2026/06/25/

 /curated/finance/monthly_revenue/

MODERNIZATION

Why the architecture kept evolving

Data lakes solved scale and openness, but they also exposed new challenges.



Benefits gained

- ✓ Multiple data sources in one platform
- ✓ Scalability and lower-cost storage
- ✓ Real-time and ML-friendly access
- ✓ Flexibility across data types



Remaining gaps

- Governance and discoverability
- Transactional reliability
- Consistent performance for BI
- Operational complexity

Data Lake



Open storage for
all data types at
scale and low cost

Governed Lake / Metadata / Serving



Add structure, quality,
security, and curated
access for reliable use

BI, Analytics, and ML



Multiple engines consume
trusted, high-quality data
consistently



These gaps led to patterns such as Data Fabric and Data Lakehouse.

COORDINATION LAYER

Data Fabric

A metadata-driven architecture that connects, governs, and delivers data across distributed systems.



What it adds

Metadata catalog, data access policies, APIs/integration, and master data management.



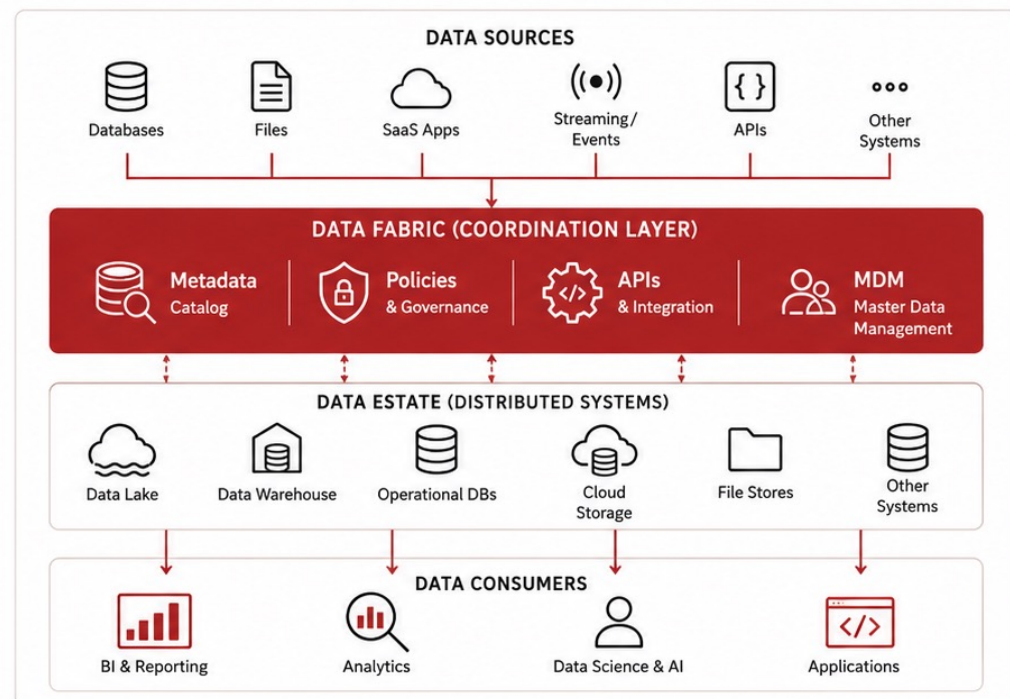
Value

Improves discoverability, policy enforcement, and cross-platform access.



Trade-offs

Can require significant integration effort, tooling maturity, and organizational discipline.



Fabric spans systems; it is not just another storage layer.

UNIFIED ANALYTICS

Data Lakehouse

Open data storage with the reliability, performance, and governance expected from a warehouse.



What it is:

A unified architecture that combines data lake flexibility with warehouse-like management features.



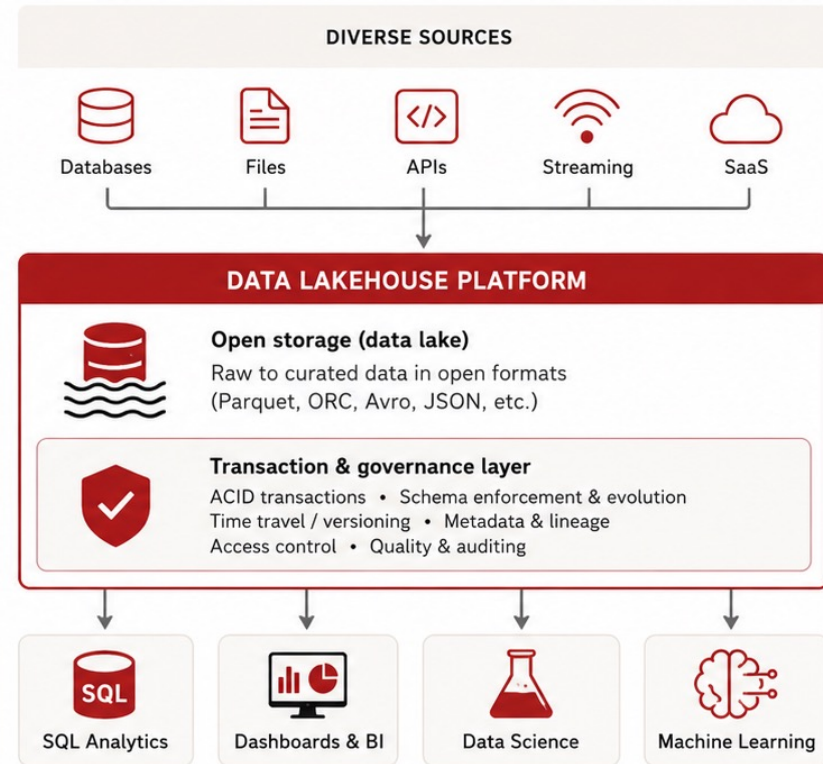
What it adds:

ACID transactions, schema enforcement and evolution, time travel/versioning, and performance optimization.



Why it matters:

Supports BI, SQL, streaming, and machine learning from one governed data platform.

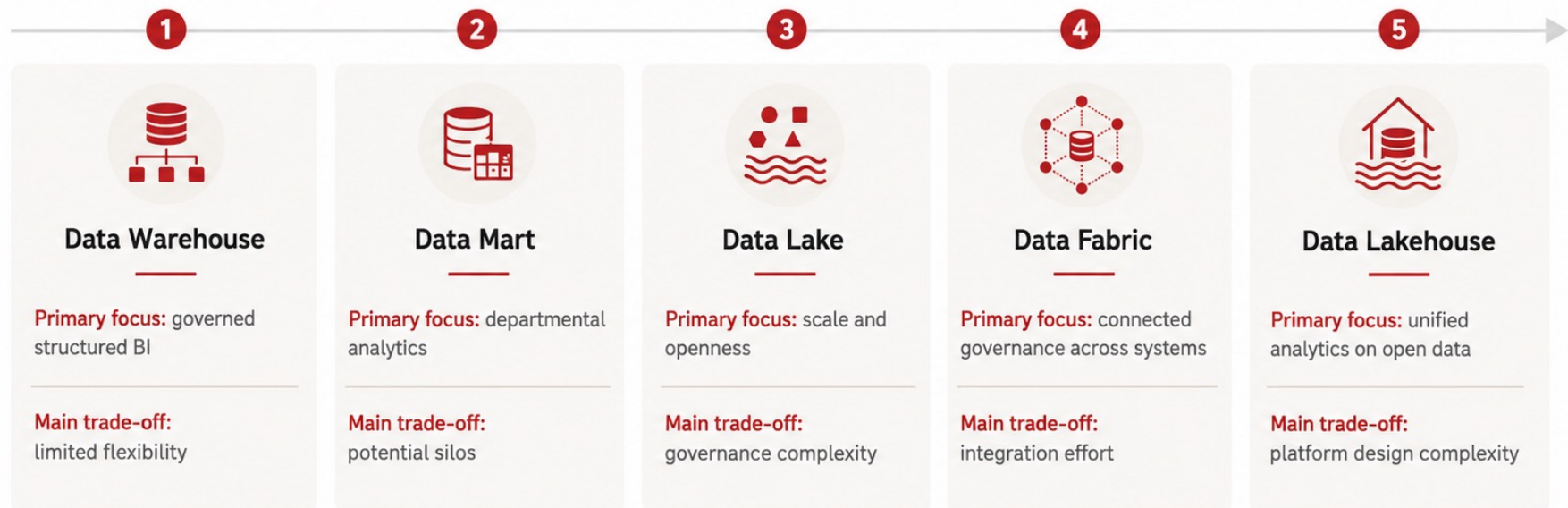


Key promise: one platform for both BI and AI workloads.

SUMMARY

Choosing the right pattern

Each architecture reflects the problems teams were trying to solve at the time.



Start from the use case: data variety, governance needs, consumer patterns, and team maturity.

ARCHITECTURE EVOLUTION

Data lake: store first, decide later

The shift

Object storage made it cheap to land raw data at scale before every future use case was known.

Strength

Accepts many formats and keeps the original evidence for replay.

Strength

Multiple engines can read the same data: SQL, Spark, ML, BI exploration.

Risk

Without ownership, metadata, quality gates, and access rules, a lake becomes a swamp.

Day 4 skills: naming, formats, zones, lifecycle, protection, and query-in-place are what keep the lake usable.

Warehouse-centered architecture: governed SQL first

What it solved

A data warehouse gives the business one curated, governed, queryable source for repeatable reporting and decision support.

Where it struggles

Raw logs, files, images, nested JSON, data-science experimentation, and very large low-value history can be expensive or awkward to force into warehouse tables first.

- Best for: trusted metrics, BI dashboards, governed SQL, dimensional models.
- The Week 3 and Week 4 deep dives: BigQuery, Snowflake, SQL, dbt, and modeling.

Lakehouse: lake flexibility + warehouse discipline

- A lakehouse adds table semantics and reliability on top of object storage.
- It keeps open file storage, but adds metadata, transactions, schema evolution, time travel, and governed access patterns.
- It exists because teams wanted one architecture for BI, data science, ML, streaming, and large-scale processing without copying data everywhere.

Now

Know the vocabulary and why it exists.

Week 5

Work with Delta Lake and Iceberg using Spark/Databricks.

Capstone

Decide which data belongs in lake, lakehouse, warehouse, or marts.

ARCHITECTURE EVOLUTION

One company uses all of them

Structure	Role in the same platform	NYC Taxi example
Data lake	Low-cost landing zone for varied raw and processed files	Raw monthly TLC Parquet + companion datasets
Lakehouse	Reliable open tables over lake files	Trips table with schema evolution and time travel
Warehouse	Curated SQL analytics and BI serving	BigQuery/Snowflake tables for demand metrics
Data mart	Focused subset for a domain or department	Operations mart for zone-hour demand dashboard

These are not maturity badges. They are different structures for different jobs.

Lake, lakehouse, warehouse, and mart

Structure	Purpose	Contents	Consumers
Data lake	Varied data at scale	Raw / processed	Engineers / science
Lakehouse	Governed reliable lake	Open tables	Eng / analyst / science
Data warehouse	Optimized SQL + reports	Curated tables	BI / business
Data mart	Domain subset	Focused metrics	Department

These structures can coexist. Choose by purpose, consumers, governance, and workload - not by trend.

DECISION POINT

One company may use all four

Lake

Land raw policy and claims files before all future uses are known.

Lakehouse

Give shared object data table metadata, snapshots, and reliable updates.

Warehouse

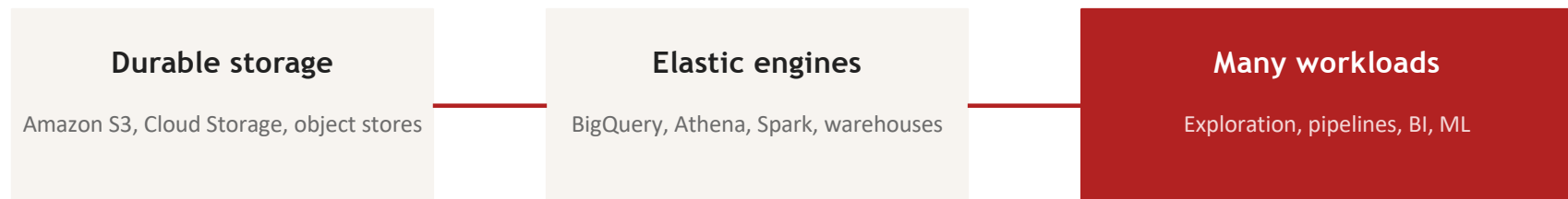
Publish governed analytical tables for repeatable SQL and BI.

Data mart

Expose a focused claims or finance view for one business domain.

CORE PRINCIPLE

Storage and compute can scale independently



- 1. Object storage provides durable, independently scalable storage.
- 2. Modern engines can read shared data and scale compute separately.
- 3. Benefits: elastic capacity, multiple engines, independent cost controls.
- 4. Responsibilities: layout, metadata, governance, quality, and scan cost.

CROSS-CLOUD TRANSLATION

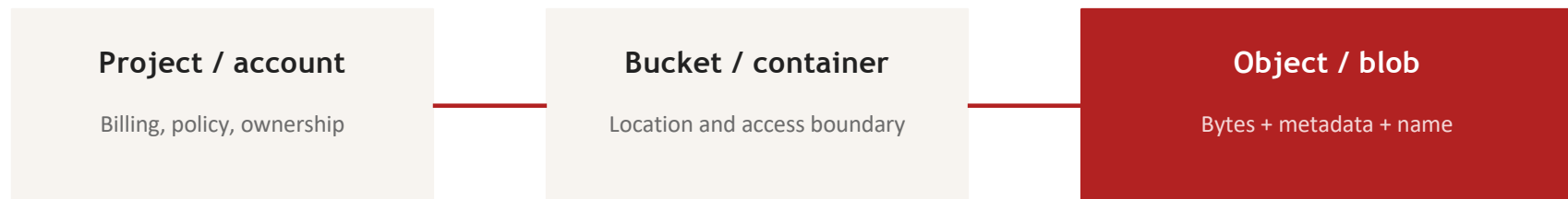
The object-storage mental model transfers

Concept	Google Cloud	AWS	Azure term
Container	Bucket	Bucket	Account + container
Stored item	Object	Object	Blob
Identifier	Object name	Object key	Blob name
URI	gs://...	s3://...	HTTPS / abfss://...

GCP and AWS remain today's equal comparison. Azure appears here only so the vocabulary is portable.

OBJECT STORAGE

Container → object → organized name



- 1. Prefixes organize names; they are not automatically separate directories.
- 2. Store CSV, JSON, Parquet, Avro, images, or documents.
- 3. Objects are accessed through APIs and URIs, not edited like disk blocks.
- 4. Naming conventions are part of the data contract.

FILE FORMATS

Choose a format for how data will be used

CSV

Human-readable rows. Simple exchange, weak types, costly repeated analytics.

JSON

Flexible nested records. Good for events/APIs, verbose and schema-variable.

Parquet

Columnar + compressed. Strong default for analytical scans.

Avro

Row-oriented binary + schema. Useful for record exchange and streaming.

ACCESS PATTERNS

Rows answer records; columns answer analytics

Row-oriented

Values for one record stay together. Efficient for whole-record reads.

Columnar

Values for one field stay together. Engines can skip unneeded columns.

CSV / JSON

Easy to inspect, but repeated parsing and broad scans add work.

Parquet

Column pruning + compression reduce I/O for analytical queries.

Compression helps; thousands of tiny files do not

Compression

Fewer bytes stored and transferred.
Codec choice trades CPU for size and compatibility.

File size

Larger analytical files reduce per-file overhead and improve parallel scan efficiency.

Small-file problem

Too many tiny files increase listings, metadata work, task setup, and cost.

Do not memorize one perfect size today. Observe the workload, then compact deliberately.

LAKEHOUSE FORMATS

File formats store bytes; table formats add transactions

File format — Parquet, ORC, Avro

How the bytes of ONE file are encoded and compressed.

Example

A single Parquet object: columnar and compressed, but with no notion of a table.

Table format — Delta, Iceberg, Hudi

A metadata layer over MANY files that behaves as one reliable table.

Adds

ACID commits, schema evolution, time travel, upserts, and safe concurrent reads.

Delta Lake vs Iceberg vs Hudi

Aspect	Delta Lake	Apache Iceberg	Apache Hudi
Origin	Databricks	Netflix → Apache	Uber → Apache
Metadata	Transaction log (_delta_log)	Manifest tree (metadata.json)	Timeline (.hoodie)
Strongest at	Databricks / Spark ecosystem	Broad multi-engine; partition evolution	Streaming upserts / CDC
Engines & catalog	Spark, Trino · Unity Catalog	Spark, Flink, Trino, Snowflake, BigQuery · REST catalog	Spark, Flink, Presto · Hive

All three add ACID, schema evolution, and time travel on cheap object storage. Hands-on with Delta + Iceberg in Week 5.

1 File Formats vs Table (Lakehouse) Formats

Two layers of your data lake

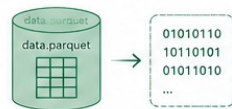
File Formats (Store Bytes)

Focus: Efficient storage & transfer

What they do:

Define how data is encoded and compressed in a single file.

Examples: Parquet, ORC, Avro



Best for:

- ✓ Scanning large datasets
- ✓ Compression & performance
- ✓ Column pruning
- ✓ Interoperability

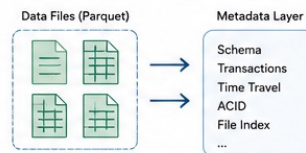
Table (Lakehouse) Formats (Add Intelligence)

Focus: Reliability, transactions & metadata

What they do:

Add a metadata layer over many files to provide table semantics.

Examples: Delta Lake, Apache Iceberg, Apache Hudi



Best for:

- ✓ ACID transactions
- ✓ Schema evolution
- ✓ Time travel & rollback
- ✓ Upserts / Deletes (MERGE)
- ✓ Consistent reads

2 Delta Lake vs Iceberg (vs Hudi at a glance)

All three bring table semantics. They differ in design choices.

Aspect	Delta Lake	Apache Iceberg	Apache Hudi
Design philosophy	Log of changes (transaction log)	Metadata tables (manifest lists)	Timeline of actions (instants)
Metadata structure	_delta_log (JSON commit files)	Manifests point to data files	Timeline + file slices (.hoodie metadata)
Strengths	Simplicity, great Python & Spark experience, broad adoption	Open spec, engine agnostic, fast metadata evolution	Streaming first, upserts at scale, built-in CDC
Time travel	Yes (versions)	Yes (snapshots)	Yes (timeline)
Upserts / Deletes	Yes (MERGE)	Yes (MERGE / DELETE)	Yes (streaming upserts)
Schema evolution	Yes	Yes (strong evolution)	Yes
Engine support	Spark, Databricks, Flink, Presto, Trino, Snowflake, BigQuery, ...	Spark, Flink, Trino, Snowflake, Athena, BigQuery, Dremio, ...	Spark, Flink, Hive, Presto, Trino, ...
Best use cases	Analytics + ML on Databricks / multi-engine	Open table format across many engines & clouds	Streaming CDC, real-time upserts, data pipelines

All three support ACID, schema evolution, and time travel on object storage.

3 How a Write Happens: Files vs Lakehouse Tables

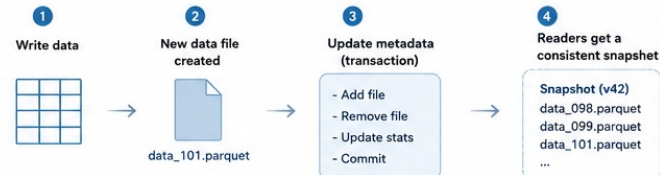
Why table formats are more than just files.

Just a File Format (e.g., Parquet)



No transactions. No schema tracking. No time travel.
Readers may see partial or inconsistent data.

Table (Lakehouse) Format (e.g., Delta Lake / Iceberg / Hudi)



Atomic commit. Consistent reads. Time travel. Schema evolution.
Upserts & deletes without rewriting everything.

★ Key Takeaway

File formats optimize how bytes are stored.

Table formats optimize how data is managed.

Use both together:

Parquet (how) +
Delta/Iceberg/Hudi (what & when).

Where it is heading: convergence on Iceberg

The standard

In 2026 Iceberg is emerging as the default: vendor-neutral, with the broadest engine support.

Convergence

Delta UniForm exposes Delta as Iceberg;
Apache XTable translates across formats;
Hudi adds native Iceberg.

In our stack

AWS S3 Tables = managed Iceberg;
Snowflake and BigQuery read Iceberg;
Databricks = Delta + UniForm.

Rule of thumb

New project? Default to Iceberg. Heavy
streaming upserts / CDC? Consider Hudi.

Mechanics and hands-on come in Week 5 Day 3: building Delta and Iceberg tables on Spark / Databricks.

LANDING ZONES

Give each data state a clear home

Raw

Original immutable arrival. Restricted access; preserve evidence and replay ability.

Quarantine

Malformed, unsafe, or policy-failing files awaiting review.

Processed

Validated, standardized, deduplicated, and efficient formats.

Curated

Consumer-ready datasets with stable meaning, ownership, and quality.

NAMING CONTRACT

Prefixes make time and state discoverable

```
# zone + dataset + date convention
raw/yellow/year=2026/month=06/day=22/trips.csv
quarantine/yellow/year=2026/month=06/day=22/bad.csv
processed/yellow/year=2026/month=06/day=22/part.parquet
curated/trips_daily/year=2026/month=06/day=22/part.parquet

# the same Hive-style keys transfer
gs://taxi-data/processed/yellow/year=2026/month=06/day=22/
s3://taxi-data/processed/yellow/year=2026/month=06/day=22/
```

Names should encode stable query dimensions, not every attribute. Avoid secrets, PII, and mutable business labels in object names.

Late, duplicate, and malformed files need policies

Late

Land in the correct event-date partition, record arrival time, and define reprocessing rules.

Duplicate

Use stable file/event identifiers, idempotent writes, and deduplication checkpoints.

Malformed

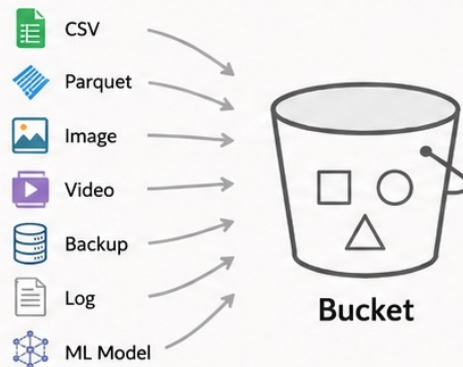
Quarantine with reason metadata; never silently promote or discard evidence.

A landing zone is reliable when exceptional data has an explicit path, owner, and recovery action.

OBJECT STORAGE

Why Object storage matters

1 What it is



BLOB = Binary Large Object.

In object storage, files are stored as objects inside buckets with metadata and a unique key.

2 Why data engineers use it

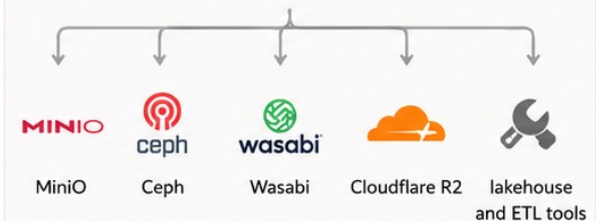
- Massive scale for files and datasets
- Low-cost, durable storage
- Decouples storage from compute
- Easy API access from many tools
- Ideal for landing zones, backups, and data lakes



3 Why S3 became the default language



Many tools speak the S3 API



This is why you often hear "S3-compatible storage" even outside AWS.



Think of object storage as the cloud file layer for modern data platforms: simple, cheap, durable, and broadly compatible.



Different providers, same concept: AWS calls it "S3"; Google calls it "Cloud Storage." Both provide object/blob storage.

DATA LIFECYCLE

Hot → warm → cold → archive

Stage	Access expectation	Typical data	Engineering decision
Hot	Frequent / low latency	Active pipeline	Keep close to compute
Warm	Occasional	Recent reruns	Lower cost; retrieval OK
Cold	Rare	Long-tail history	Accept fees or delay
Archive	Exceptional	Legal / historical	Optimize retention cost

“Frozen” is informal. Cloud storage classes use provider-specific names. Tier by access, retrieval tolerance, retention, and total cost.

PROTECTION CONTROLS

Recovery, automation, and compliance differ

Control	Primary purpose	Behavior / use
Lifecycle	Automate tier or deletion	Cost + housekeeping
Soft delete	Recover recently deleted objects	Timed recovery window
Versioning	Keep older object generations	Overwrite/delete recovery
Retention	Block deletion before an age	Minimum retention
Bucket Lock	Make retention irreversible	Approved compliance
Holds	Protect selected objects	Event/legal workflow

Choose the failure or policy you must address. Recovery controls do not replace retention, and lifecycle does not enforce compliance.

Storage classes: S3 and GCS side by side

Access need	AWS S3	Google Cloud Storage	Notes
Hot / frequent	S3 Standard	Standard	No minimum duration
Auto-tiering	S3 Intelligent-Tiering	Autoclass	Moves objects for you
Warm / infrequent	S3 Standard-IA / One Zone-IA	Nearline	~30-day minimum
Cold / archive	S3 Glacier Instant / Flexible / Deep Archive	Coldline / Archive	90–365-day min; GCS stays instant-access

Key difference: every GCS class has instant (ms) access; S3 Glacier Flexible and Deep Archive add minutes-to-hours retrieval latency.

Same controls, different mechanics

Capability	AWS S3	Google Cloud Storage
Lifecycle	Transition + expire; abort multipart; expire noncurrent	SetStorageClass + Delete by age / version
Versioning	S3 Versioning (+ optional MFA delete)	Object Versioning + soft delete (7-day default)
Geo-redundancy	Single-region bucket + CRR / SRR replication	Native region / dual-region / multi-region buckets
WORM / compliance	Object Lock: Governance & Compliance + Legal Hold	Retention policy + Bucket Lock + object holds
Access control	IAM + bucket policies; Block Public Access	IAM + uniform bucket-level access; Public Access Prevention
Holds	Protect selected objects	Event/legal workflow

Same concepts on both clouds. Biggest mechanical gap: S3 buckets are single-region (replicate for geo); GCS offers native multi-region buckets.

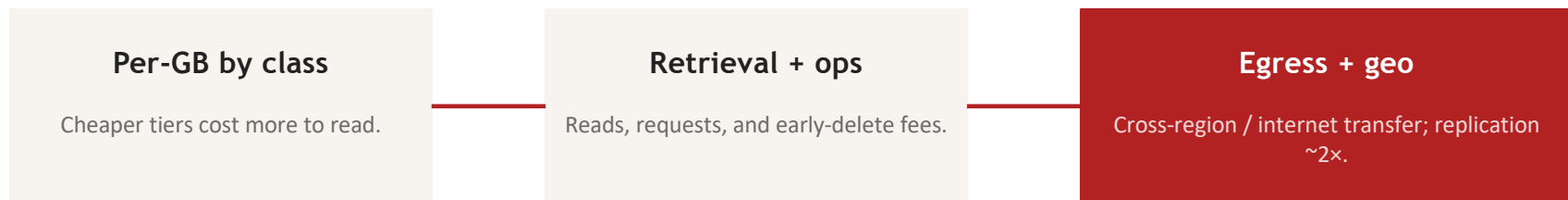
S3 BUCKET TYPES

Four bucket types — use General purpose

Bucket type	What it does	When to use it
General purpose	Original all-purpose S3: any object, all storage classes, multi-AZ	Landing zones & data lakes — our default
Directory	Single-AZ, ultra-low latency (S3 Express One Zone)	Hot, latency-critical, high-request workloads
Table	Managed Apache Iceberg tables (S3 Tables)	Analytics / ML tabular data — see Week 5
Vector	Embedding vectors + similarity-search indexes	AI / ML vector search — see Week 6

Pick General purpose for object-storage and data-lake work. Directory, Table, and Vector buckets are specialized — out of scope today.

Storage cost: the same five levers on both clouds



- 1. Bill on five levers: per-GB by class, retrieval fees, request/operation costs, data egress, and minimum-duration / early-delete charges.
- 2. Match the tier to the access pattern, not just age — cold tiers are cheap to store, costly to read.
- 3. Multi-region and replication buy resilience at roughly double storage plus egress; choose it deliberately.
- 4. Same meter habit as Day 2 (BigQuery bytes scanned) and Day 3 (Codespaces core-hours).

SCHEMA

Schema-on-read vs. schema-on-write

Schema-on-read

Store first; apply declared or inferred structure when an engine reads.

Strength

Fast exploration, flexible sources, multiple interpretations.

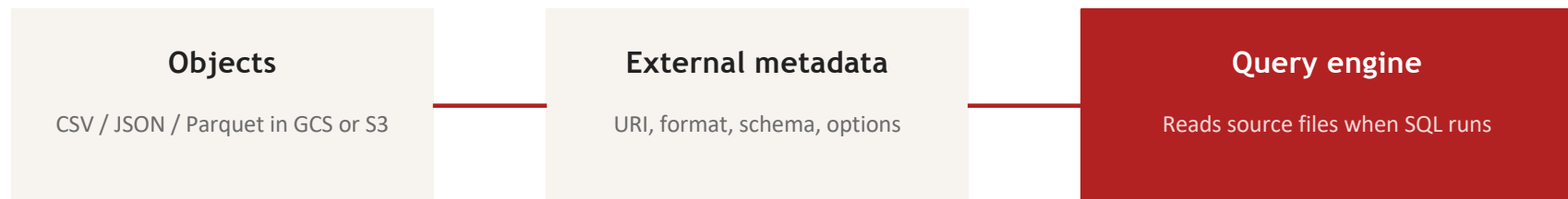
Schema-on-write

Validate and shape data before publishing it to the target table.

Strength

Stable contracts, predictable BI, stronger quality enforcement.

External tables connect SQL to stored files



- 1. Useful for exploration, validation, infrequent access, and shared data.
- 2. Scan behavior and source format still affect cost and performance.
- 3. External data may have consistency or feature limitations.
- 4. Load curated native tables for repeated, governed, performance-sensitive BI.

1 Automated Data Discovery with Crawlers & Catalogs

Crawlers scan your object storage, infer schema/metadata, and populate a Data Catalog.



Amazon S3
Data in object storage (CSV, JSON, Parquet, etc.)



AWS Glue Crawler
Scans and infers schema, partitions, and metadata



AWS Glue Data Catalog
Central metadata repository of tables, schemas, and partitions



Used by many services



... and more

Key Benefits

- ✓ No manual schema definition
- ✓ Automatically handles new partitions and files
- ✓ Centralized metadata for analytics & ETL
- ✓ Enables self-service data discovery



Google Cloud Storage
Data in object storage (CSV, JSON, Parquet, etc.)



Dataplex Crawler
Scans and infers schema, partitions, and metadata



Dataplex Data Catalog
Central metadata repository (entries, schemas, partitions)



Used by many services



... and more

Key Benefits

- ✓ No manual schema definition
- ✓ Automatically handles new partitions and files
- ✓ Centralized metadata for analytics & ETL
- ✓ Enables self-service data discovery



Crawlers read file formats, sample data, and folder structure to infer schemas and partitions, keeping your catalog up to date automatically.

2 Query Data in Object Storage (At the Source)

Query data directly where it lives—no loading or copying required. Serverless engines read the data, apply schema, and return results.



Amazon S3
(Data)

Read data
in place



Amazon Athena
(Serverless SQL)



Query Results
(S3)

How it works

- 1 Write data to S3 in open formats (CSV, JSON, Parquet, ORC).
- 2 Athena reads the data from S3 using the Glue Data Catalog (or built-in).
- 3 Run ANSI SQL queries and get results without moving the data.

Benefits

- ✓ No data movement
- ✓ Pay per query (serverless)
- ✓ Scales to petabytes
- ✓ Works with open formats



Cloud Storage
(Data)

Read data
in place



BigQuery
(Serverless SQL)



Query Results
(BigQuery / GCS)

How it works

- 1 Write data to Cloud Storage in open formats.
- 2 BigQuery reads the data from GCS (external tables or native tables).
- 3 Run standard SQL and get results without loading the data.

Benefits

- ✓ No data movement
- ✓ Pay per query (serverless)
- ✓ Scales to petabytes
- ✓ Enterprise security & governance



Best for exploratory analysis, ad-hoc queries, dashboards, and data you don't want to move. Combine with catalogs (Glue/Dataplex) for easier discovery and governance.

Discover, then query – the same two layers on both clouds

Layer	AWS	Google Cloud	What it does
Object storage	Amazon S3	Cloud Storage (GCS)	Where the files actually live
Schema discovery / catalog	Glue Crawler → Glue Data Catalog	BigQuery auto-discovery; Knowledge Catalog (formerly Dataplex)	Infers schema, registers tables for you
Serverless query engine	Amazon Athena	BigQuery (external / BigLake)	Runs SQL over the files in place
Define a table by hand	CREATE EXTERNAL TABLE	CREATE EXTERNAL TABLE (autodetect)	The manual alternative to discovery

Same pattern on both clouds: a catalog holds the table definition; a serverless engine queries the files in place. Discovery tools just fill the catalog for you.

Does Athena need Glue? It needs a metastore

Way to define the table	What it is	Reach for it when	Effort
Manual CREATE EXTERNAL TABLE	You write the DDL → Glue Data Catalog	Known, stable schema; a few tables	You type it
Glue Crawler	Auto-infers schema → catalog	Many folders, changing schema, new partitions	Automated (recommended at scale)
Partition Projection	Athena derives partitions from a pattern	Heavily partitioned data	Config, no crawl
Athena Federation	Query RDS / DynamoDB / etc. directly	Data is not in S3 at all	Connector

Athena needs a metastore (the Glue Data Catalog), not a crawler. A crawler is the automated way to fill it — best at scale and change; for a few known files, hand-written DDL is simpler.

QUERY ENGINES

SQL brought to the files in place

Engine	What it is	Where you meet it
Presto / Trino	Open-source distributed SQL engines (Presto forked into Trino)	The lineage behind Athena
Amazon Athena	Managed, serverless Trino/Presto querying S3 in place	Today's AWS lab (Activity 4)
BigQuery / BigLake	Google's serverless engine; BigLake queries GCS in place	Activity 2 (GCP)
Spark SQL	Distributed engine for large-scale ETL and SQL	Week 5
DuckDB · Dremio · Redshift Spectrum	Other in-place engines (local, lakehouse, S3)	Awareness

The pattern matters more than the brand: a query engine separates compute from storage and brings SQL to files in place. Athena is essentially managed Trino/Presto.

REQUIRED LAB

Build a secure Cloud Storage landing zone

Goal - create, inspect, organize, protect, and explain a private GCS landing zone.

100 min

1. Verify account, project, billing, APIs, and instructor readiness.
2. Create private raw/processed buckets; upload and organize sample objects.
3. Predict and observe authenticated vs incognito access behavior.
4. Apply lifecycle, inspect soft delete, enable versioning, overwrite, and restore.
5. Submit Q1-Q7, evidence, lifecycle JSON, and control explanations.

Fallback: pair with a ready learner or use the embedded evidence. Do not attach personal billing.

REQUIRED ACTIVITIES

Query files, then design the storage contract

Goal - connect stored objects to SQL, then defend an operable NYC Taxi layout.

85 min

1. Query-in-place lab (40 min): external table, supplied SQL, actual bytes processed, limitations.
2. Team design (45 min): raw/quarantine/processed/curated zones and date prefixes.
3. Define formats, late/duplicate/malformed handling, access, and lifecycle.
4. Choose one query-in-place use and one warehouse use; prepare a short readout.

Deliverable: individual query notes plus one team storage convention for the Day 5 pipeline.

Carry the storage contract into Day 5

- 1 Object storage is the durable foundation; formats, layout, metadata, controls, and compute make it useful.
- 2 Tomorrow's NYC Taxi pipeline lands into explicit zones, handles bad data, and promotes trusted outputs.
- 3 Exit: explain one difference between a data lake and a data warehouse without naming a product.
- 4 Exit: give one benefit and one new responsibility created by separating storage and compute.
- 5 Exit: choose one accidental-deletion control and one compliance-retention control; explain the difference.